

Ros Day01 hw3 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.09

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 과제에서는 ROS2 의 turtlesim 을 이용하여 키보드 입력에 따라 삼각형, 사각형, 원을 그리는 C++ 패키지를 제작하였다.

과제를 진행하면서 ROS2 C++ 패키지의 파일 구조를 구성하는 방법, Publisher 와 Subscriber 를 사용하는 방법, turtlesim 에서 제공하는 Topic 과 Service 를 사용하는 방법을 이해하기 위해 AI 를 활용하였다.

AI 에게 과제 조건을 바탕으로 프로그램의 전체적인 구조와 필요한 기능을 질문하였고, 작성된 코드가 어떤 역할을 하는지 설명을 들으면서 구현하였다.

또한, 단순히 코드를 작성하는 것뿐만 아니라 빌드 과정에서 발생한 오류와 도형이 정확하게 그려지지 않는 문제를 해결하는 과정에서도 AI 를 활용하여 문제를 해결하였다.

2. AI 사용 부분

2.1 Publisher 를 이용한 거북이 이동

거북이를 움직이기 위해 /turtle1/cmd_vel Topic 에 geometry_msgs/msg/Twist 메시지를 보내야 했는데, 처음에는 Publisher 가 어떤 역할을 하는지 정확히 이해하지 못하였다.
AI 의 설명을 참고하여 다음과 같이 Publisher 를 생성하였다.

```
cmd_vel_pub_ =  
    this->create_publisher<geometry_msgs::msg::Twist>(  
        "/turtle1/cmd_vel", 10);
```

이를 이용하여 삼각형과 사각형에서는 직진과 회전을 반복하고, 원에서는 직진 속도와 회전 속도를 동시에 주어 원운동을 하도록 구현하였다.

이 부분에서 Topic 통신은 Publisher 가 데이터를 보내고 Subscriber 가 데이터를 받는 구조라는 것을 실제 동작을 통해 이해할 수 있었다.

2.2 삼각형과 사각형이 찌그러지는 문제

처음 작성한 프로그램에서는 정해진 시간 동안 속도를 주는 방식으로 이동과 회전을 구현하였다.

예를 들어 일정 시간 동안 직진한 뒤 일정 시간 동안 회전하는 방식이었다.

프로그램 자체는 동작하였지만 삼각형과 사각형을 여러 번 그려보니 각도가 조금씩 어긋나면서 도형이 찌그러지는 문제가 발생하였다.

AI 에게 해당 문제를 질문한 결과, 시간만 이용하여 이동과 회전을 제어하면 실행 과정에서 발생하는 작은 오차가 반복되면서 누적될 수 있다는 설명을 들었다.

이를 개선하기 위해 ai 의 도움을 받아 /turtle1/pose Topic 을 Subscriber 로 받아 거북이의 실제 위치와 방향을 확인하도록 수정하였다.

```
pose_sub_ =
```

```

this->create_subscription<turtlesim::msg::Pose>(
    "/turtle1/pose",
    10,
    std::bind(
        &TurtleDrawer::poseCallback,
        this,
        std::placeholders::_1));

```

Pose 메시지에서 x, y, theta 값을 받아 현재 위치와 방향을 확인하고 목표 거리 또는 목표 각도에 도달하면 거북이를 멈추도록 변경하였다.

수정 후에는 이전보다 삼각형과 사각형의 모양이 정확하게 그려지는 것을 확인하였다.

또한 추가적으로 도형을 전부 그리고 나면 도형을 다시 그릴 수 있게 1 초 뒤 삭제하는 기능 또한 AI의 도움을 받아 추가하였다.

turtlesim의 /clear Service를 이용하면 기존에 그려진 선을 지울 수 있다는 것을 알려주었고, std_srvs/srv/Empty를 사용하여 구현하였다

잘 되긴 하는데 삼각형이랑 사각형이 너무 찌그러짐

그리고 그렸던건 다 그린 후에 1초있다 지워지게 하면 안됨? 다시 그리게



52s 동안 처리함

○○. 둘 다 고치는 게 낫다.

지금 삼각형/사각형이 찌그러지는 가장 큰 이유는 “2초 동안 움직여라 / 몇 초 동안 돌아라”처럼 **시간만 보고 제어**하고 있어서 실행할 때마다 작은 오차가 누적되기 때문이야. 아까 과제 1에서 회전값이 매번 미묘하게 달라졌던 문제랑 비슷한 계열이다.

이번에는 숫자를 또 찍어 맞추지 말고 `/turtle1/pose` 에서 실제 거북이 위치와 각도를 받아서, 목표 거리·각도에 도달하면 멈추게 바꾸자. 그리고 도형이 완성되면 1초 보여준 뒤 `/clear` 서비스로 **선만 삭제**하면 된다. 다음 키를 누르면 다시 깨끗하게 그릴 수 있어.

PDF도 Turtlesim이 제공하는 토픽 메시지를 활용하라고 요구하므로, `/cmd_vel` 을 publish하면서 `/pose` 까지 사용하는 구조는 과제 취지에도 잘 맞는다.  ROS2_day1_intern21th

오류 원인은 간단해. 아직 `parkdoyeon_hw3` 라는 ROS2 패키지를 만든 게 아니라서 그래.

지금 화면을 보면 현재 위치가

```
</> Bash
~/colcon_ws/src
```

인데 여기서 바로

```
</> Bash
mkdir -p include/parkdoyeon_hw3
```

를 실행했어. 그래서 지금 실제로는 대충 이런 상태야.

```
colcon_ws/
├── src/
│   └── include/
│       └── parkdoyeon_hw3/
│           └── turtle_drawer.hpp
```

그런데 `src/turtle_drawer.cpp` 를 만들려고 하니까 현재 `~/colcon_ws/src` 안에는 `src` 라는 폴더가 없어서

```
touch: cannot touch 'src/turtle_drawer.cpp': No such file or directory
```

가 뜬 거야.

지금 이렇게 고치자

먼저 방금 잘못 만든 `include` 폴더를 지워도 된다.

```
</> Bash
cd ~/colcon_ws/src
rm -rf include
```

그리고 패키지부터 생성해야 해.

```
</> Bash
ros2 pkg create parkdoyeon_hw3 \
--build-type ament_cmake \
--dependencies rclcpp geometry_msgs turtlesim
```



3. 깨달은 점

처음에는 ROS2 의 Topic 과 Service 의 차이를 정확히 이해하지 못했지만 실제 프로그램을 구현하면서 `/turtle1/cmd_vel` 은 계속해서 속도 데이터를 보내는 Topic 이고, `/turtle1/set_pen`, `/clear`, `/turtle1/teleport_absolute` 등은 특정 기능을 요청하는 Service 라는 차이를 이해할 수 있었다.

또한 Publisher 는 메시지를 보내는 역할을 하고 Subscriber 는 메시지를 받는 역할을 한다는 것도 AI 를 통해 알게 되었다. 특히 /turtle1/pose 데이터를 Subscriber 로 받아 거북이의 실제 위치와 방향을 이용해 제어하면서 Subscriber 가 단순히 데이터를 출력하는 용도뿐만 아니라 실제 제어에도 사용될 수 있다는 것을 알게 되었다.

처음에는 일정 시간 동안 속도를 주면 원하는 각도와 거리만큼 정확하게 움직일 것이라고 생각하였다. 하지만 실제 실행에서는 작은 오차가 발생하고, 삼각형이나 사각형처럼 같은 동작을 반복하면 그 오차가 계속영향을 주는 것을 확인하였고 이를 통하여 실제 상태를 확인하면서 제어하는 방식이 더 정확할 수 있다는 것을 알게 되었다.